



These 16 DSA Patterns Did What 3000 LeetCode Problems Couldn't



[Himanshu Singour](#)

Follow

14 min read

.

Jun 19, 2025

Listen

Share

More



When I started with DSA, I did what everyone else does.
I opened LeetCode. Picked Easy.
Clicked “Run”. Then “Submit”.
Got a green tick. I felt smart.

But in interviews? I choked. Repeatedly.

I had solved over **3000 problems** across platforms from LeetCode to Codeforces.

Still, the moment someone asked me a real-world DSA question, my mind went blank.

I wasn't stupid. I just didn't have structure.

I was solving 3000 problems like they were 3000 random puzzles.

What I **needed** was to see the **pattern** behind the problem.

And once I discovered that everything changed.

16 DSA patterns that finally made me stop memorizing and start solving.

1. Sliding Window, When You Don't Want to Repeat the Same Work

What Happened:

I got this question

"Find the maximum sum of any subarray of size k ."

I wrote two loops:

- First loop to pick the start point
- Second loop to add the next k elements

It worked. But when the array got bigger, my code became slow. It was checking too much again and again.

What I Realized:

Why am I calculating the same thing again?

If I already know the sum of a window, and I'm just moving 1 step forward I can:

- Subtract the element going out
- Add the new element coming in

That's called **Sliding Window**.

When to Use:

- When you're working with arrays or strings
- When the question asks about a **range** or **subarray**
- And the size of that range is fixed (or sometimes flexible)

How It Works:

Let's say we have:

```
Array: [1, 2, 3, 4, 5], Window size k = 3
```

- First window: $1 + 2 + 3 = 6$
- Next window: Instead of adding $2 + 3 + 4$ again, we do: $6 - 1$ (remove first) + 4 (add next) = 9
- Keep doing this till end

Now you're only doing work once, and just adjusting.

Why It's Amazing:

Because you're using your **previous answer** smartly.

No more doing the same math again and again.

It saves a lot of time goes from $O(n*k)$ to just $O(n)$.

2. Two Pointers, When You Want to Check from Both Sides

What Happened:

I had to check if a word is a palindrome (same forward and backward).

I used to reverse the string and compare.

But that's extra work and takes more memory.

What I Realized:

Why not just check from both sides?

Use one pointer at the beginning, and one at the end and move them toward each other.

When to Use:

- When you're working with arrays or strings
- When you want to compare from both ends
- Great for palindrome, pair-sum, reversing, etc.

How It Works:

Example: "madam"

- Compare 1st and last: `m == m`
- Then 2nd and 2nd last: `a == a`
- Then middle: `d` done

Another use:

```
Array: [1, 2, 3, 4, 6], Target = 6
```

- Start = 1, End = 6 $\rightarrow 1+6 = 7 \rightarrow$ too big $\rightarrow$ move end left
- Start = 1, End = 4 $\rightarrow 1+4 = 5 \rightarrow$ too small $\rightarrow$ move start right
- Start = 2, End = 4 $\rightarrow 2+4 = 6$ Found!

Why It's Powerful:

Because you're not checking every possible pair you're moving smartly based on conditions.

No need for two loops. Just two fingers sliding inward.

3. Fast and Slow Pointers, When You Want to Catch a Cycle

What Happened:

I was asked:

“How do you find if a linked list has a cycle?”

I used a `Set` to store visited nodes.

It worked, but it used extra memory.

What I Learned:

There's a smarter way use two pointers:

- One moves slowly (1 step)
- One moves faster (2 steps)

If there is a loop, they will meet.

If not, the fast one will reach the end.

When to Use:

- When checking if something loops or cycles
- Mostly in linked lists or circular paths
- Also used in some array problems

How It Works:

Example:

```
1 -> 2 -> 3 -> 4 -> 2 (back to node 2 = cycle)
```

Steps:

- slow = 2, fast = 3
- slow = 3, fast = 2
- slow = 4, fast = 4 → same → cycle found

Why It's Smart:

It uses **no extra space**

And it's super fast just two pointers running at different speeds.

You catch the cycle like one person chasing another on a circular track.

Eventually, they'll meet.

4. Prefix Sum, When You Keep Getting "Find the Sum from X to Y" Type Questions

My Struggle:

I had this question:

“Find the sum of elements from index 2 to 5.”

I did what most beginners do I used a loop every time to sum from index 2 to 5. It worked... but then I had to do it 100 times. And suddenly my code was **slow**.

The Realization:

Why am I doing the same sum again and again?

That’s when I found this trick called **Prefix Sum**.

When to Use:

- When the problem asks: “Sum from index i to j ?”
- Or: “How many times something happened up to index k ?”
- Best for arrays where you do **many range sum queries**

How It Works:

You build a new array called `prefix[]` where:

```
prefix[i] = sum of all elements from 0 to i
```

Let’s take a simple array:

```
arr = [1, 2, 3, 4, 5]
```

Now build prefix sum:

```
prefix = [1, 3, 6, 10, 15]
```

Now, to get sum from index 2 to 4 (which is $3 + 4 + 5 = 12$):

Just do:

```
prefix[4] - prefix[1] = 15 - 3 = 12
```

Why It's Helpful:

Because instead of looping every time to calculate a sum you can now answer it in just **1 subtraction**.

It changes your code from **$O(n)$** to **$O(1)$** per query.

And if you get 100 such queries?

Your total time goes from $100 * n \rightarrow$ just 100

5. Binary Search, When You Need to “Search Smart”

My Struggle:

I thought binary search was only for finding a number in a sorted array. So I used it once or twice and ignored it after that.

But later I realized it's way more powerful than that.

You can use binary search to **optimize**, **minimize**, or **maximize** things.

When to Use:

- When the array is **sorted**
- Or when you're told to “find the minimum/maximum value” that satisfies a condition
- Or any time you're dealing with **searching in a range**

How It Works:

You start with a range: `low` and `high`.

You keep checking the `mid` value.

Depending on the condition, you either:

- Move to the **left half** (if `mid` is too big)
- Or move to the **right half** (if `mid` is too small)

Simple Example, Classic Search:

```
arr = [1, 3, 5, 7, 9], target = 5
```

- `mid = 5` → found!

Now let's see a non-obvious example:

Example, Square Root of 10

You want to find the square root of 10 up to 2 decimal places.

Try $\text{mid} = (1 + 10) / 2 = 5.5$

→ $5.5 * 5.5 = 30$ → too high → move left

Keep narrowing the range. Finally, you'll reach something like 3.16

This is binary search but on **answers**, not elements.

Why It's Powerful:

Because it cuts the search space in **half every time**.

So even if your range is 1 to 1 billion you'll find the answer in just ~30 steps.

6. Two Heaps, When You Need a Live Median or Top k Elements

My Struggle:

I got this problem:

“Keep adding numbers and find the median after each add.

I was totally stuck.

How do I keep track of the middle element every time?

I tried sorting the list after every insertion. That was too slow.

The Realization:

Use two heaps:

- One **max heap** for the smaller half of numbers
- One **min heap** for the bigger half

When to Use:

- When the input is coming in **live / stream**
- And you want to find the **median**, or **top K numbers**
- Used in real-world apps like stock prices, live score tracking

How It Works:

Let's say you get numbers: 1, 2, 3, 4, 5

Step by step:

- Add 1 → goes to left heap → median = 1

- Add 2 → goes to right heap → median = $(1+2)/2 = 1.5$
- Add 3 → left: [1,2], right: [3] → balance → median = 2

You keep both heaps **balanced**, so that median is easy to find:

- If total count is odd → median is top of bigger heap
- If even → average of both tops

Why It's Useful:

Because it gives you the answer in real-time, without sorting again and again.

Also very efficient: insertions take $O(\log n)$, median takes $O(1)$

7. Bit Manipulation, When You Want to Use Math Magic

My Struggle:

I got this question:

“In an array where every number appears twice except one, find the number that appears only once.

I tried using a HashMap to count frequencies. It worked... but wasn't very efficient.

Then I saw a one-line solution using something called **XOR**.

I didn't understand it at first. But once I did, it felt like **math + magic**.

When to Use:

- When question talks about **even-odd appearances**

- Or if it says “every number appears twice except one”
- Or if you want to check even/odd, or swap numbers without extra space

How It Works:

The trick here is **XOR** ($\wedge$) a bitwise operation.

It follows a few cool rules:

- $a \wedge a = 0$
- $a \wedge 0 = a$
- $a \wedge b \wedge a = b$ (because $a \wedge a = 0$, and $0 \wedge b = b$)

So if we XOR all the numbers:

```
arr = [2, 3, 2]
```

Do:

$$2 \wedge 3 = 1$$

$$1 \wedge 2 = 3 \rightarrow \text{so the answer is } 3$$

Why It's Cool:

You don't need extra space or loops.

You just scan the array once and get the answer.

Bit manipulation is also used in:

- Checking if a number is even: $\text{num} \& 1 == 0$
- Dividing/multiplying by 2: $\text{num} \ll 1$ (multiply), $\text{num} \gg 1$ (divide)

- Counting set bits: `num & (num - 1)`

Pro Tip:

It sounds hard at first, but once you try a few problems, it becomes really fun like doing puzzles with switches.

8. Dynamic Programming (DP), When Your Code Keeps Repeating the Same Work

My Struggle:

I got this classic question:

“You can climb 1 or 2 steps at a time. How many ways to reach the top if there are `n` steps?”

I used recursion:

```
ways(n) = ways(n-1) + ways(n-2)
```

It worked... until `n = 35` and my code became super slow.

Why? Because it was calculating the same thing **again and again**.

The Realization:

In Dynamic Programming, you **don't recalculate**. You **store answers** (called “memoization”).

When to Use:

- If the problem breaks into **smaller subproblems**
- And the same subproblem is being solved **many times**
- Words to look for: “maximum”, “minimum”, “ways to do something”

How It Works:

Let's take the **climbing stairs** example again.

For $n = 5$, you want to know:

```
ways(5) = ways(4) + ways(3)
```

So you just store results for:

```
ways[1] = 1  
ways[2] = 2  
ways[3] = 3  
...
```

You build from bottom up:

```
for (i = 3 to n):  
    dp[i] = dp[i-1] + dp[i-2]
```

No repetition. Fast. Clean.

Real-Life Examples:

- Longest Increasing Subsequence
- Knapsack (bag full of items)
- Coin change
- Palindromic substrings
- Edit distance (word conversions)

Why It's Powerful:

Because it takes a slow, repeating solution... and makes it fast.

From $O(2^n) \rightarrow O(n)$

Pro tip:

Always ask: "Am I solving the same small problem again and again?"
if yes, it's probably DP.

9. Backtracking, When You Have to Try Everything But Not Stupidly

My Struggle:

I got a question:

"Generate all possible combinations of well-formed parentheses.

I tried brute-force made all strings of (and) and then checked which ones are valid.

That worked, but it created **way too many strings**.

Then I learned about **Backtracking** a way to explore all possibilities, but smartly.

When to Use:

- When you need to find **all combinations**
- Or generate **valid sequences**
- Or **explore paths** in a maze, tree, or board

How It Works:

Backtracking is like:

- Try a choice
- Go forward
- If it fails, go **back** and try a different choice

It's like solving a maze:

You try a path if it hits a wall, you go back and take another.

Example, Parentheses

You want to generate all valid pairs of 2 brackets:

Start with empty string: ""

- Try adding (→ "("
- Try adding (again → "(("
- Can't add (anymore (limit reached), so try) → "(() "
- Keep going
- If invalid, backtrack and change path

Other Famous Examples:

- N Queens
- Sudoku Solver
- Word Search on a grid
- Subsets / Permutations / Combinations

Why It's Beautiful:

Because you don't try everything blindly you **prune bad choices early**.

Think of it like:

“Try something, go deep. If it doesn't work, undo and try something else.”

10. DFS & BFS, When You're Exploring a Grid, Graph, or Connections

My Struggle:

I got a problem that said:

“Count the number of islands in a grid of 0s and 1s.

I was totally confused. What even is an island?

How do I find which 1s are connected?

I thought maybe I should scan the whole grid again and again. But then I saw a simple solution using **DFS**.

When to Use:

- Grid or graph problems
- When you need to **explore all connected things**
- Questions like: “Find all paths”, “Count components”, “Spread fire”, “Flood fill”, etc.

How It Works:

DFS (Depth First Search):

- Goes as deep as possible first, like diving straight into one path.
- Usually uses **recursion** or a **stack**

BFS (Breadth First Search):

- Explores level by level, like throwing waves.
- Usually uses a **queue**

Example, Number of Islands

You have:

```
grid = [  
    [1,1,0],  
    [1,0,0],  
    [0,0,1]  
]
```

There are 2 islands:

- One at top-left (all connected 1s)
- One at bottom-right (single 1)

You pick any unvisited **1**, and **spread** using DFS or BFS to mark all connected 1s.

Real Use-Cases:

- Maze solver
- Google Maps path finding
- Word Search

- Finding friend groups in social networks
- Fire spreading in forest simulations

Why It's Important:

Because many real-world systems (like social networks, cities, mazes) can be modeled as graphs.

Once you know DFS and BFS any grid or network-type problem becomes easy to think through.

11. Union Find, When You Want to Group Things Without Loops

My Struggle:

I saw a problem:

“Check if adding a connection creates a cycle.

I had no idea how to do this without building a full graph.

Then I learned about **Union Find** (also called Disjoint Set Union or DSU).

It's like a smart way to keep track of which things belong together.

When to Use:

- When you need to **group things**
- When the question says: “Are A and B connected?”
- When checking for **cycles** in undirected graphs

How It Works:

Each element starts in its own group.

You use two functions:

1. **Find:** What is the “leader” (parent) of this group?
2. **Union:** Combine two groups into one

Use a **parent[]** array for this.

Example:

Let’s say:

```
Connections = [ (1, 2), (2, 3), (4, 5), (3, 1) ]
```

You start by making:

```
parent[1] = 1  
parent[2] = 2  
parent[3] = 3  
parent[4] = 4  
parent[5] = 5
```

Now:

- Union(1, 2) → now 2 follows 1
- Union(2, 3) → now 3 follows 1
- Now try Union(3, 1) → both already connected → **cycle**

That’s how you catch a cycle!

Why It's Useful:

Because it's fast and memory-efficient.

You don't need full graph logic.

You just track **which group each item belongs to**.

Used in:

- Kruskal's Algorithm (Minimum Spanning Tree)
- Friend circles / social groups
- Connected components in networks

12. Topological Sort, When One Thing Must Happen Before Another

My Struggle:

I was asked:

“You have some courses to complete. Some courses need other courses to be done first. Find a valid order to complete all.

This confused me. I tried brute-force, but it didn't work.

Then I learned this amazing trick: **Topological Sort**

When to Use:

- When you're given **dependencies** or “A must come before B”
- Usually for **scheduling, task order, course planning**
- You're working with a **Directed Acyclic Graph (DAG)**

How It Works:

You build a graph where:

- Each node is a task
- An edge from $A \rightarrow B$ means “A must come before B”

Then use:

- **Kahn’s Algorithm (BFS-based)**
- **OR DFS with stack** to build the order

Example, Course Schedule

Let’s say:

```
Courses: A, B, C  
Prerequisites: A  $\rightarrow$  B, B  $\rightarrow$  C
```

Valid order: $A \rightarrow B \rightarrow C$

Steps:

- Build graph
- Count how many incoming edges each node has (called indegree)
- Start with courses having **0 prerequisites**
- Process them, reduce indegrees of their children
- If any node’s indegree becomes 0, add it to the queue

Why It's Powerful:

Because you can turn a mess of tasks into a clean order as long as there's **no cycle**.

It's used in:

- Build systems (like compiling code)
- Course planners
- Job/task schedulers
- Dependency managers (npm, Maven)

13. Greedy Algorithm, When You Pick the Best Step Right Now and Trust It Works Out

My Struggle:

I got a question:

“You're given start and end times of meetings. What's the maximum number of meetings you can attend without overlaps?”

I thought: “Let's try all combinations.”

But that was super slow.

Then I read about **Greedy Algorithms** make the **best local choice** at each step, and you'll often get the best global result.

When to Use:

- When you can sort things and **decide one-by-one**
- When there's a clear “best” or “lowest cost” at each step
- When you need to **maximize** or **minimize** something

How It Works:

Greedy means:

“At every step, pick the option that looks best at that moment.

You **don't backtrack**. You trust your choice and move forward.

Example, Meeting Rooms

Meetings:

```
[ (1, 4), (2, 5), (4, 7), (6, 8) ]
```

Sort by end time:

```
[ (1, 4), (4, 7), (6, 8), (2, 5) ]
```

Start picking:

- Pick (1, 4)
- Next is (4, 7) → doesn't overlap
- Then (6, 8) → overlaps →
- Done!

Answer: 2 meetings

Why It Works:

Because you **don't need to try all options** you just make smart choices using sort + loop.

Other examples:

- Activity selection
- Fractional knapsack
- Jump game
- Gas station problems

14. Stack / Monotonic Stack When You Need to Track Previous or Next Bigger/Smaller Thing

My Struggle:

I got this question:

“For each day, tell me how many days to wait until a warmer temperature.”

My first try: nested loops compare today with every day after. Too slow.

Then I saw this new idea **Monotonic Stack**.

When to Use:

- When you're asked to find:
“next greater”, “previous smaller”, etc.
- When you need to **track a history** (like brackets, changes, or reversals)
- Common in array problems

How It Works:

You keep a **stack** of useful values.

Let's say you want to find the **next greater number** for each element.

```
arr = [2, 1, 3]
```

You push 2, then 1 — when you see 3, it's greater than both, so:

- 1 → next greater = 3
- 2 → next greater = 3
- 3 → no greater after it

Use stack to maintain values in **decreasing order** (monotonic).

Classic Use Cases:

- Next Greater Element
- Daily Temperatures
- Histogram Largest Rectangle
- Valid Parentheses
- Undo actions / editor history

Why It's Useful:

Because it **remembers what came before** in a smart way.
And avoids brute-force checking every pair.

15. Trie When You Want to Store or Search Words Quickly

My Struggle:

I was trying to build an autocomplete feature.

I used an array or HashMap of words, and did “startsWith()” search.

It worked... but it got **slow** with 1000s of words.

Then I found **Trie (prefix tree)** a tree where **each level is one letter**.

When to Use:

- When you’re dealing with a **dictionary of words**
- And you need to **insert, search, or autocomplete** efficiently
- Any problem with prefixes

How It Works:

You build a tree where:

- Each node is a letter
- Paths form words
- Every word ends with a special “end” marker

Example:

Insert “cat” and “cap” into trie:



Now:

- Search “cat”? → yes
- Search “cab”? → no
- StartsWith “ca”? → yes

Why It’s Amazing:

Because it **shares letters** between words, so it’s fast and memory-friendly.

Used in:

- Word suggestions
- Spell checking
- Phone contact search
- Google-like autocomplete

16. Kadane’s Algorithm, When You Need the Max Sum Subarray

My Struggle:

I was given:

“Find the maximum sum of any subarray in a list of numbers (positive and negative).

I used two loops. Check every subarray. $O(n^2)$.
Slow again.

Then I found **Kadane’s Algorithm** and it blew my mind.

When to Use:

- When you want to find:
- Maximum subarray sum
- Maximum profit (buy/sell)
- Especially with **positive and negative numbers**

How It Works:

You go through the array once.

At each step:

- Either continue the current subarray
- Or start fresh from this number

You keep tracking:

- `currentSum = max(num, currentSum + num)`
- `maxSum = max(maxSum, currentSum)`

Example:

```
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

Best subarray: `[4, -1, 2, 1]` → sum = 6

Kadane finds it in $O(n)$ time only one loop!

Why It Works:

Because you **don't reset everything on a bad number**.

You just decide whether to continue or restart smartly.

One of the most elegant algorithms in DSA.

Final Thoughts: Pattern > Problem

When I started, I used to open a problem and just **stare**.

Now, I look and say:

“This is a sliding window problem”

“Oh, looks like two pointers can solve this”

“This is definitely backtracking with pruning” ...and just like that, I know how to begin.

Learning these 16 patterns changed everything for me.

Data Structures

Data Structure Algorithm

Algorithms

Software Engineering

Engineer